Section: CS OCY, OC1

FixProof: A Validation-Oriented System for AI-Generated Vulnerability Remediation

Tony Tran

Progress Report 5 - Submission Candidate, October 25, 2026

Problem Statement:

AI coding tools can propose security repairs that appear plausible but leave a weakness exploitable, change intended behavior, or satisfy one evaluator while failing another. Disappearance of a static-analysis finding alone is therefore incomplete evidence of a correct repair. Pearce et al. (2022) found security weaknesses in a substantial portion of Copilot-generated programs within their selected scenarios, while later research has examined LLM vulnerability repair and the role of compiler, sanitizer, and test feedback (Pearce et al., 2023; Kulsum et al., 2024).

FixProof asks a narrower question: can separately recorded static, runtime security, functional, deterministic-policy, and human-review evidence support a defensible decision about an AI-generated patch? The evaluation uses three purpose-built JavaScript/Express applications containing reflected XSS (CWE-79), SQL injection (CWE-89), and path traversal (CWE-22). Five scheduled repair calls were made per fixture. These 15 calls are repeated repairs of three applications, not 15 independent applications and not a production repair-success sample.

The project translates beyond this small benchmark through its evidence and authority structure rather than through its measured success percentages. To use FixProof on another repository, an evaluator would need repository-specific build commands, scanner configuration, target-finding identity, security and benign-behavior oracles, and a documented human-review rule. The transferable claim is that these evidence channels should be preserved separately and that the repair model should not approve its own candidate.

Solution Statement:

FixProof is a Python orchestration prototype that records a normalized Semgrep finding, constructs a focused remediation prompt, obtains candidate code, applies it in a copied workspace, and runs syntax, candidate SAST, targeted runtime-security, and functional checks. A deterministic policy routes each candidate to REJECT, READY_FOR_HUMAN_REVIEW, or NEEDS_HUMAN_ADJUDICATION. Human verdicts and rationales are stored separately from automated outcomes. The system does not automatically approve or deploy repaired code.

Here, "Python orchestration" means that Python invokes the defined stages, passes artifact paths between them, and writes structured evidence. Python is not treated as an independent security judge. The registered validators produce measurements, the deterministic policy routes those measurements, and a person makes and documents any acceptance or follow-up decision.

The design choices remain deliberate. Minimal fixtures provide reproducible ground truth and benign-behavior expectations within the course timeline, but they sacrifice production realism. A fixed generator, prompt template, tool setting, and five-call schedule hold generation conditions constant without creating independent application samples. Copied workspaces protect frozen inputs but are not security sandboxes. Static and runtime checks answer different questions, and later evidence qualifies earlier conclusions without rewriting the original study history.

The public website is a read-only evidence reference. It presents saved, sanitized results, payloads, requests, responses, decisions, and rationales. It does not call a model, scan code, execute attacks, approve a patch, or deploy software.

Completed Tasks Since Progress Report 4:

• Finalized, reviewed, and posted Video IV and submitted the combined Video IV and Peer Feedback Report IV assignment on October 22. The main seven-slide presentation explains the Windows-inconclusive PATH-S06 case, the separately frozen WSL2 extension, its zero-pass/five-fail result, the later human qualifications, and the limitation that one registered case does not establish application-wide security.

- Refined the Video IV terminology so the intentionally vulnerable original application is labeled as a positive control whose disclosure confirms that the Linux symbolic-link fixture works. It is not described as a security pass and is excluded from the five-candidate denominator.
- Prepared terminal-level backup evidence for Traversal 01-05. Each manual spot-check verified the saved candidate source hash, normal-file response, direct parent-traversal rejection, and PATH-S06 symbolic-link disclosure. These presentation checks do not add registered observations. The formal extension human metadata continues to identify only Traversal 02 as the originally recorded manual reproduction.

- Corrected the Netlify package after the first Windows-created ZIP stored nested paths with backslashes and produced HTTP 404 responses. Version 2 uses forward-slash archive paths and places index.html at the archive root.
- Published the corrected package at https://fixproof.netlify.app/ and performed a dated live validation on October 22. The home page, test-evidence page, JSON evidence files, workbook, appendix, scripts, and workflow image were available. Five downloaded evidence artifacts matched the local release files by SHA-256.
- Ran the existing Playwright validation against the live HTTPS deployment. It confirmed 15 primary rows, 15 supplemental rows, 245 case-evidence rows, 29 human-decision records, candidate details, pagination, and the registered filters. There were no application page errors or unexpected console errors. Four content-security-policy messages came from optional Netlify hosting injections and did not affect the application.
- Completed a full clean-package reproduction rehearsal from a disposable Git snapshot. The process created a new Python environment, installed pinned Python dependencies and all six Node application dependency trees, installed the Playwright browser, verified recorded evidence, and ran the controlled demo suite.
- Passed all 131 automated tests during readiness verification and again during the guided demo suite. The verifier confirmed 15/15 primary initial attempts, 10/10 original conflict reviews, evidence/control separation, and artifact bindings.
- Reproduced four controlled candidate outcomes: SQLi ready for human review, XSS rejection for functional regression, XSS human adjudication for SAST/runtime disagreement, and path-traversal human adjudication for SAST/runtime disagreement. The separated non-AI false-success control was also verified.
- Recorded an additional dependency-maintenance limitation. All npm ci operations completed from the committed lockfiles, but npm reported existing moderate, high, and critical advisories in the fixture dependency trees. I did not run npm audit fix, because changing those lockfiles would silently alter the frozen benchmark. The advisories will be disclosed and triaged separately from the experimental results.
- Prepared the Video V outline and recording script as a final-project checkpoint. The presentation traces the progression from the initial pilot, through primary-v1, supplemental-v1, and PATH-S06, to the clean reproduction and final deliverable structure without adding a new experiment.

Response to Prior Instructor Feedback and Draft Work:

Progress Report 3 instructor feedback dated October 12 asked me to share code snippets and output as draft work. I responded here with a small, traceable implementation excerpt and a saved-result excerpt rather than a large code listing. The complete source and evidence remain in the repository and public workbook. The intended reading order is: Python coordinates the stages; validators record results; policy routes a candidate; human review supplies a separate verdict and rationale.

The following excerpt from src/fixproof/primary_trials.py shows the primary orchestrator calling the security and functional validators and then giving their saved files, together with the preliminary syntax/SAST file, to the decision engine:

```
security = run_security_validation(
    contexts_file=context_file,
    workspace_dir=workspace_directory,
    canonical_id=preparation["canonical_id"],
    output_file=security_file,
)
functional = run_functional_validation(
    contexts_file=context_file,
    workspace_dir=workspace_directory,
    canonical_id=preparation["canonical_id"],
    output_file=functional_file,
)
decision = run_decision_engine(
    preliminary_file=preliminary_file,
    security_file=security_file,
    functional_file=functional_file,
    output_file=decision_file,
)
```

This code does not label a patch "secure." It produces separate artifacts and passes them to the policy. For example, when runtime security and functional checks passed but the target Semgrep finding persisted, the saved primary policy result was NEEDS_HUMAN_ADJUDICATION, not approval.

The following selected fields from the frozen PATH-S06 summary show the shape of recorded output. Ellipses mark omitted candidate rows and path/SHA-256 binding fields that remain in the JSON record. The candidate response was a failure because HTTP 200 disclosed the controlled outside-file marker:

```
"candidate_counts": {
  "pass": 0,
  "fail": 5,
  "inconclusive": 0
},
...
{
  "target_id": "primary-v1-path-traversal-initial-02",
  "status": "fail",
  "marker_disclosed": true,
  "status_code": 200,
  ...
}
```

The source record is data/extensions/path-s06-v1/runs/20260930T180925912688Z-path-s06-v1/summary.json. The vulnerable baseline was a positive control used to confirm that the symbolic-link test setup could expose the marker; it is not counted as one of the five candidate passes or failures.

Effort and Weekly Contribution:

From October 18 through October 25, I spent two hours per day, for 16 hours total. The time below is self-reported and covers review, documentation, organization, feedback work, validation, and presentation preparation; it does not represent new model calls or new registered experimental observations.

Date	Hours	Work completed	Contribution to final deliverables
October 18	2	Reviewed the saved XSS, SQLi, and traversal candidate instances and their evidence layers	Rechecked how primary, supplemental, and later human records relate without overwriting one another
October 19	2	Reviewed request/response outcomes and existing human rationales	Improved consistency between recorded test results and the explanation presented to reviewers
October 20	2	Reviewed and organized coursework, evidence, and final-deliverable folders	Made supporting artifacts easier to locate and reduced ambiguity between working files and frozen evidence
October 21	2	Reviewed PATH-S06 evidence and refined the Video IV and corrected Netlify materials	Communicated the symbolic-link result and corrected the public package's path handling
October 22	2	Validated the live evidence site and completed the clean-package reproduction rehearsal	Confirmed deployed artifact bindings, interactive evidence views, 131 tests, and controlled demo behavior
October 23	2	Added source-linked Python orchestration and saved-output excerpts to Report 5	Responded directly to instructor feedback and clarified the boundary between coordination, validation, policy, and human review
October 24	2	Reviewed documentation, result wording, file organization, and feedback communication	Improved clarity for student and instructor review while preserving the frozen denominators
October 25	2	Finalized the Report 5 submission candidate and prepared the Video V outline and script	Connected the completed evaluation and reproducibility work to the remaining final milestones
Total	16	Two hours per day for eight days	Progress Report 5, Video V preparation, and final-deliverable readiness

Open Items and Next Milestone Work:

- Summarize any substantive feedback received on FixProof with an adopt, defer, or decline disposition for the final report and presentation. Feedback is communication evidence and does not change the frozen experiment records.

- Decide how to present the npm audit advisories. The current recommendation is to preserve the frozen benchmark lockfiles, report the advisories as a maintenance limitation, and create a separately versioned dependency-update plan only if time permits.
- Convert the existing final-report placeholder into a self-contained working draft containing methodology, analysis, results, limitations, conclusions, reproducibility, and AI-use disclosure.
- Create the first final release manifest with selected artifacts, file sizes, SHA-256 values, environment versions, and explicit exclusions.

Tasks for the Next Project Report or Final Milestone:

- Incorporate accepted Video IV feedback into the final report and final presentation while preserving the registered evidence and denominators.
- Record and post Video V from the prepared outline as a concise final-project checkpoint focused on reproducibility, claim-to-evidence mapping, and final deliverable status rather than introducing another broad experiment.
- Draft the final report's results and limitations sections using the separate primary-v1, supplemental-v1, and PATH-S06 extension tables.
- Build the final presentation around the workflow figure, the primary candidate matrix, one XSS parity example, the PATH-S06 example, the reproducibility result, limitations, and conclusions.
- Generate and verify the exact final submission package, public access link, manifest, and checksums. Test permissions in a private browser and retain a local archival copy of the submitted version.

Questions I Have or Issues I Am Running Into:

The largest remaining methodological issue is not another missing test; it is how to communicate the limits of the evidence without weakening its value. The project shows that separately preserved validators and human review can expose disagreement, regressions, robustness gaps, and a symbolic-link failure within three controlled Express fixtures. It cannot estimate production repair reliability or establish scanner accuracy across repositories.

The clean reproduction also surfaced npm dependency advisories. These are not candidate-test outcomes and should not be mixed with the repair evaluation. Updating the dependencies inside the frozen benchmark could change behavior and invalidate direct reproduction. The defensible approach is to disclose the advisories, keep the evaluated snapshot immutable, and treat any remediation as a separate maintenance version.

Feedback requested: Is the distinction between a reproducible research snapshot and a production-ready software release clear? For the final presentation, which evidence is most persuasive: the clean reproduction, the candidate-level matrices, or a focused request/response example?

Methodology Paragraph Summary:

Each controlled fixture defines an attacker-controlled input, a vulnerable operation, synthetic data, a baseline attack, and benign-behavior expectations. Primary-v1 used five scheduled initial repair calls per CWE while holding the generator (gpt-5.2), prompt template (1.0), and model tool access (none) constant. The model received the target finding and focused source context but not the evaluator's hidden runtime tests. Saved candidates received syntax, Semgrep, targeted HTTP/browser security, and functional checks. Deterministic policy and separate human review then interpreted the recorded evidence.

Supplemental-v1 was registered before candidate execution and applied new security, behavioral-parity, and robustness contracts to the same 15 saved candidates. It made no new model calls and did not rerun primary SAST. The later PATH-S06 extension registered one Linux symbolic-link case, reused the baseline and five saved traversal candidates, and preserved supplemental-v1's five Windows-inconclusive outcomes. Manual presentation spot-checks and the public dashboard are verification and communication activities, not additional experimental observations.

The clean-package rehearsal used a disposable release-candidate commit and the existing scripts/verify-clean.ps1 process. It exported the snapshot with Git, created a new environment, restored dependencies from project metadata and committed lockfiles, recomputed readiness checks, ran the 131-test suite, and executed controlled demonstrations in disposable directories. No original model call was repeated and no registered evidence was overwritten.

OWASP WSTG and Semgrep's run-rules documentation remain technical references for runtime-test organization and scanner configuration. They are not treated as experimental datasets. The public website and workbook are sanitized views of repository records and are not execution environments.

Timeline:

When	Description of Task	Status
September 29	Submit Progress Report 3	Completed early; due October 4
September 30	Freeze and execute PATH-S06 extension; record five human qualifications	Completed
October 1	Build student evidence workbook and request/response tables	Completed
October 6	Submit Progress Report 4	Completed early; due October 18
October 21	Build and locally validate corrected Netlify PATH-S06 version 2 package	Completed
October 21-22	Finalize and post Video IV	Completed
October 22	Publish and browser-validate live Netlify evidence reference	Completed
October 22	Complete clean-package reproduction and controlled demo suite	Completed
October 18-25	Review evidence, organize artifacts, refine documentation, and prepare Report 5/Video V	Completed; 16 hours
October 22	Submit Video IV and Peer Feedback Report IV	Completed early; combined assignment due October 25
November 1, 11:59 p.m.	Submit Progress Report 5	Planned per supplied course schedule
November 3	Post Video V	Planned per supplied course schedule
November 8, 11:59 p.m.	Complete Peer Feedback Report V	Planned per supplied course schedule
November 15, 11:59 p.m.	Submit final presentation, no more than 15 minutes	Planned per supplied course schedule
November 22, 11:59 p.m.	Submit peer feedback on final presentation	Planned per supplied course schedule
December 6, 11:59 p.m.	Submit final report and reproducible supporting package	Planned per supplied course schedule

Evaluation:

The registered experimental results remain unchanged. Publishing the website, running manual presentation checks, and rehearsing the package did not add model calls or change any experimental denominator.

Primary case	Initial attempts	Target SAST resolved	Security pass	Functional pass	Automated decision
Reflected XSS	5	0/5	5/5	5/5	5 need adjudication
SQL injection	5	5/5	5/5	5/5	5 ready for review
Path traversal	5	0/5	5/5	5/5	5 need adjudication
Total	15	5/15	15/15	15/15	5 ready for review; 10 need adjudication

Security and functional columns count candidates whose complete registered primary suite passed. They are not individual test counts. Target-SAST resolution was 5/15, while 10/15 candidates showed SAST/runtime disagreement. These are descriptive results for the controlled fixtures, not estimates of Semgrep accuracy or production repair success.

Evidence layer	Category	Observations	Pass	Fail	Inconclusive
Primary-v1	Security	40	40	0	0
Primary-v1	Functional	60	60	0	0
Supplemental-v1	Security	60	55	0	5
Supplemental-v1	Behavioral parity	45	40	5	0
Supplemental-v1	Robustness	35	25	10	0
PATH-S06 extension-v1	Security	5	0	5	0

The 100 primary observations, 140 supplemental-v1 observations, and five extension observations remain separate. They must not be pooled into a single security rate. The public evidence table exposes all 245 rows but retains an evidence-layer field and the original category-specific decision rules.

Human evidence layer	Records	Current interpretation
Original primary conflict reviews	10	Seven acceptances and three requests for more testing under original evidence
Supplemental-v1 later decisions	14	Five rejections, five bounded SQLi acceptances, and four requests for more testing
PATH-S06 extension qualifications	5	Five later traversal rejections; prior records preserved

Human evidence layer	Records	Current interpretation
Total displayed human records	29	Separate authorities and dates; not one approval rate

The live deployment validation confirmed that these values and their evidence bindings are available through the public dashboard and workbook. The clean reproduction recomputed the repository's readiness checks and authoritative demo decisions from a clean exported snapshot. These activities increase confidence in traceability and reproducibility, but they do not add application diversity or prove production readiness.

Report 5 checkpoint	State on October 25
Video IV	Posted; combined assignment submitted October 22
Peer Feedback Report IV	Submitted October 22 with the combined Video IV assignment
Progress Report 5	Submission candidate prepared; 16 hours recorded for October 18-25
Video V	Outline and recording script prepared; recording/posting remains
Corrected Netlify package	Published and live-validated
Public evidence	15 candidates; 245 case rows; 29 human records
PATH-S06 extension	Frozen separately at 0 pass, 5 fail, 0 inconclusive
Clean-package rehearsal	Passed; fresh dependency install, evidence verification, 131 tests, and controlled demo suite
Dependency advisories	Disclosed maintenance limitation; frozen lockfiles unchanged
Primary-v1 and supplemental-v1	Frozen and unchanged
Final report	Structured placeholder exists; full self-contained draft is the next writing milestone
Final release manifest	Planned

Claim-to-Evidence Map:

Claim	Supporting evidence	Boundary
FixProof preserves multiple validation signals	Primary, supplemental, policy, and human records	Three controlled Express fixtures
Static and runtime evidence can disagree	10/15 primary candidates	Not a general scanner-accuracy estimate
Exact behavioral parity can qualify a security repair	Four XSS-P01 supplemental failures and later rationales	Frozen missing-input contract only
The five saved traversal candidates fail PATH-S06	Five HTTP 200 responses disclosing the controlled marker	One registered symbolic-link case
Summary counts are traceable	245-row table, workbook, JSON evidence paths, and rationales	Saved evidence; website does not rerun tests
The package is reproducible in the tested environment	Clean export, fresh dependency install, 131 tests, evidence verification, and demo suite	Same snapshot and documented environment; not independent replication

Strength of Evidence and Claim Boundary:

I use qualitative evidence strength for a specific claim rather than a statistical confidence percentage. The five repairs per fixture are repeated calls against the same application, so they are not independent samples from a larger population. "Strong within the registered scope" means that a defined oracle produced consistent, conclusive evidence for the named cases. It does not mean that the result generalizes to production applications or untested payloads.

Claim being assessed	Evidence strength	Basis	Important limit
Registered XSS payloads were blocked	Strong within registered scope	All five candidates passed the primary security suite and all 15 supplemental XSS security observations passed	One reflected-XSS fixture and a fixed payload set; four candidates later failed missing-input parity
The XSS candidates fully conformed to all supplemental expectations	Mixed	XSS 03 passed all nine supplemental cases; XSS 01, 02, 04, and 05 failed XSS-P01 and were later rejected for bounded parity	The rejection is about the frozen missing-name behavior, not continued XSS exploitability
Registered SQL-injection attacks were blocked	Strong within registered scope	All five candidates passed primary security, 15/15 supplemental security observations, and 10/10 parity observations with no unauthorized rows	One SQLi fixture; the five saved sources were identical and each failed the separate SQL-R01 input-handling contract

Claim being assessed	Evidence strength	Basis	Important limit
Direct parent-segment traversal was blocked	Strong for the registered direct-input cases	The saved candidates rejected the registered direct ../ request and passed the executed primary/supplemental traversal security checks	Lexical containment did not establish symbolic-link target containment
The saved traversal candidates remained vulnerable to PATH-S06	Strong within the one registered extension case	The frozen WSL2 run produced five conclusive failures: HTTP 200 and marker disclosure for all five candidates	One Linux symbolic-link topology; not a claim about every traversal technique or platform
FixProof establishes a production repair-success rate	Not established	The benchmark supports traceable case-level observations and reproducibility, not population inference	Three controlled applications and repeated repairs cannot support a production-wide percentage

Report Outline:

1. Introduction and bounded research question.
2. Related work and technical references.
3. FixProof architecture and separation of authority.
4. Primary-v1 protocol and evidence.
5. Supplemental-v1 protocol and evidence.
6. Separately versioned PATH-S06 extension.
7. Human-review methodology and decisions.
8. Draft implementation/output evidence, evaluation, claim-to-evidence mapping, and reproducibility.
9. Limitations, dependency-maintenance boundary, and threats to validity.
10. Conclusions and future work.
11. AI-use disclosure, references, and supporting-material index.

References:

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. 2022 IEEE Symposium on Security and Privacy. https://doi.org/10.1109/SP46214.2022.9833571

Pearce, H., Tan, B., Ahmad, B., Karri, R., & Dolan-Gavitt, B. (2023). Examining zero-shot vulnerability repair with large language models. 2023 IEEE Symposium on Security and Privacy. https://doi.org/10.1109/SP46215.2023.10179324

Kulsum, U., Zhu, H., Xu, B., & d'Amorim, M. (2024). A case study of LLM for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback. Proceedings of the 1st ACM International Conference on AI-Powered Software. https://doi.org/10.1145/3664646.3664770

OWASP Foundation. (n.d.). Web Security Testing Guide, version 4.2. https://wstg.owasp.org/v4.2/

Semgrep. (n.d.). Run rules. https://semgrep.dev/docs/running-rules

Appendix:

The frozen primary protocol and evidence are under docs/study-protocol-v1.md, benchmarks/primary/v1/, data/primary_trials/v1/, data/primary_reviews/v1/, and data/evaluation/primary-report.json. Supplemental-v1 is defined by docs/supplemental-protocol-v1.md and tests/supplemental/v1/cases.json; its evidence remains under data/supplemental/v1/ and totals 140 observations.

The separate PATH-S06 extension is defined by docs/path-s06-extension-v1-protocol.md, hash-locked in data/extensions/path-s06-v1/protocol-lock.json, and summarized in docs/path-s06-extension-v1-results.md. Its five candidate outcomes are not part of supplemental-v1's 140-observation denominator.

The student evidence workbook is docs/coursework/review-2026-10-01/FixProof-15-candidate-test-evidence.xlsx. The public reference is available at https://fixproof.netlify.app/. The live deployment record and screenshots are under docs/coursework/review-2026-10-22/, together with the clean-package reproduction validation.

AI-use disclosure: ChatGPT/Codex assisted planning, coding, testing, presentations, and drafting; OpenAI API calls generated candidates. I validated the evidence and remain responsible for decisions and submitted work.